LAB 05 | GitOps Under Pressure

Kubernetes, Argo CD, infrastructure drift and security policy | 2 engineers | 3 h 20 min core

Scenario

You are two platform engineers brought in after a team migrated CityBike from a local Docker pipeline to Kubernetes. Their service is deployed by Argo CD. Someone modifies Kubernetes manually, a service configuration change breaks production access, and security policy must be tightened without losing availability.

Unlike the previous lab, you will NOT build a Docker image, promote through five environments, or repeat a canary rollback. This task examines reconciliation, Kubernetes networking, health checks and declarative policy.

Learning outcomes

- Explain desired state, live state, drift and Argo CD self-healing.
- Diagnose Kubernetes Service-to-Pod routing and distinguish synchronization from application health.
- Fix an incident by changing Git (not by applying ad hoc production patches).
- Enforce the Restricted Pod Security Standard while preserving the app.
- Defend a design decision with command output and evidence.

Teams, roles and rules

Student A — Platform	Student B — SRE / Security
Own Git repository, Application manifest, Argo synchronization	Own probes, smoke test, diagnosis and security acceptance
Lead Missions 0–2; review partner changes later	Lead Missions 3–5; review partner changes later

Swap keyboard and primary engineer halfway through. Both students must author at least one commit and review at least one change. Use a throwaway public repository containing only lab code. Never commit secrets. The `kind-citybike-gitops` context must be confirmed before changing cluster state.

Timebox — 200 minutes

Clock	Mission	Acceptance evidence
0:00–0:20	0 · Setup & preflight	Tools pass; local cluster identified
0:20–0:55	1 · Desired state & Argo CD	Application Synced/Healthy; smoke succeeds
0:55–1:20	2 · Infrastructure drift	Manual replica change self-heals
1:20–1:55	3 · Git-delivered outage	Service failure diagnosed, Git fix restored
1:55–2:35	4 · Restricted security policy	New pods admitted, hardened, healthy
2:35–3:05	5 · Readiness failure & recovery	Unready rollout diagnosed, Git reverted
3:05–3:20	6 · Defense	Evidence bundle and short oral review

Prerequisites (complete before the timed lab)

- Docker Desktop/Engine, kind, kubectl, Git, curl, Bash-compatible shell, internet access. Node.js 22+ is optional for local unit tests.
- At least 8 GB RAM allocated to the Docker environment is recommended for kind + Argo CD.
- A public GitHub repository is needed for the GitOps exercise; Argo CD must be able to fetch it.
- Extract the complete CityBike starter project, including `gitops/`, `bootstrap/`, `scripts/`, `tests/` and `docs/`.

```
bash scripts/preflight.sh
kubectl config current-context    # check before using any kubectl command
# Read docs/SETUP.md and publish the starter to your own PUBLIC throwaway Git repo.
```

Mission 0 — Safe cluster setup (20 min)

Create or select the disposable kind cluster and install Argo CD using the official manifest. The commands are in docs/SETUP.md. Do not touch a workplace or shared production context.

```
kind create cluster --name citybike-gitops --config kind-config.yaml
kubectl config current-context
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Deliverable M0: screenshot or output confirming the current context and running Argo CD repository server. If Argo is preinstalled by the instructor, use the supplied cluster and skip installation.

Mission 1 — Deploy declared desired state (35 min)

Read `gitops/kustomization.yaml`, `deployment.yaml` and `service.yaml` before deploying. Explain how Kustomize turns the app source into a versioned ConfigMap, and why a ConfigMap name hash can trigger a rollout.

Publish the starter to your own public GitHub repo, customize `bootstrap/application.template.yaml` into ignored `bootstrap/application.yaml`, and apply it. Allow Argo CD to synchronize automatically.

```
kubectl apply -f bootstrap/application.yaml
kubectl -n argocd get applications.argoproj.io citybike
kubectl -n citybike get deploy,pods,svc
bash scripts/smoke-service.sh
```

Questions: Why are the Kubernetes manifests the source of truth? What does Synced mean? Can Synced coexist with a broken user experience? Deliverable M1: Application Synced/Healthy, 2 ready replicas, successful /api/stations smoke.

Mission 2 — Catch infrastructure drift (25 min)

Student A plays an operator who makes a manual scaling change in the live cluster. Student B must catch the divergence and demonstrate automatic reconciliation without manually restoring the replicas.

```
kubectl -n citybike scale deployment/citybike --replicas=0
kubectl -n citybike get deployment/citybike -w
```

Watch Argo CD and Kubernetes. Desired Git state is 2 replicas. If needed, request a hard refresh of the Argo Application. Record a before/after timeline. Why does `kubectl scale` not persist when Argo self-healing is enabled? What would happen if selfHeal were disabled?

Mission 3 — A bad Git change breaks access (35 min)

Create branch `feature/broken-service`. In `gitops/service.yaml`, intentionally change `targetPort: 3000` to `targetPort: 3999`. Submit the change through a peer review, then push/merge to main to make it the desired state. Do not fix the live Service with kubectl.

```
kubectl -n argocd get applications.argoproj.io citybike
kubectl -n citybike get pods,svc,endpointslice
kubectl -n citybike describe svc citybike
bash scripts/smoke-service.sh    # failure is expected
```

Investigate: Why can the pods remain Ready while the Service is unreachable? Does Argo Synced guarantee that clients are served successfully? Identify the precise routing mismatch and document the evidence.

Restore access by reverting the Git change (new commit to main) and waiting for Argo to reconcile. Deliverable M3: failed smoke, root cause, successful smoke after Git-based recovery.

Mission 4 — Security admission without downtime (40 min)

The original Deployment lacks explicit container hardening. You must bring it into compliance with the Kubernetes Restricted Pod Security Standard. Work ONLY through a Git change and a peer review.

Add appropriate pod/container security contexts: non-root execution, RuntimeDefault seccomp, deny privilege escalation, drop capabilities, and a read-only root filesystem when compatible. Do not remove the probes or change the application features.

```
kubectl label namespace citybike pod-security.kubernetes.io/warn=restricted --overwrite
kubectl label namespace citybike pod-security.kubernetes.io/audit=restricted --overwrite
# Commit/push the hardened Deployment. Wait for rollout.
kubectl -n citybike rollout status deployment/citybike --timeout=180s
# Only AFTER successful rollout, turn on enforcement:
kubectl label namespace citybike pod-security.kubernetes.io/enforce=restricted --overwrite
# Change APP_RELEASE in Git, commit, push; let Argo reconcile.
kubectl -n citybike rollout status deployment/citybike --timeout=180s
```

Acceptance: new Pods are admitted under Restricted policy, 2 replicas Ready, smoke passes. Explain which controls mitigate which risks. Do not use privileged mode or disable admission to make the test pass.

Mission 5 — Readiness is not liveness (30 min)

Induce a declarative readiness failure: in `gitops/deployment.yaml`, set the `READY_MODE` environment variable to `fail` and push to main. Observe rollout status, pod readiness and service reachability. Because maxUnavailable=0, old healthy pods may continue serving; capture what actually happens.

```
kubectl -n citybike rollout status deployment/citybike --timeout=75s
kubectl -n citybike get pods
kubectl -n citybike describe deployment citybike
# Fix via a new Git commit restoring READY_MODE=ok.
```

Compare `/health/live` and `/health/ready`. Why is keeping them separate a reliability improvement? Demonstrate a healthy Git-reconciled rollout after the fix.

Mission 6 — Final engineering defense (15 min)

Evidence required	Proof
GitOps deployment	Repository URL; Argo Application Synced & Healthy
Drift and self-healing	Before/after replica count and reconciliation evidence
Service outage	Bad targetPort; error screenshot/log; Git fix and passing smoke
Security	Hardened Deployment diff; namespace Restricted label; ready new Pods
Readiness challenge	Failed rollout evidence and healthy recovery
Collaboration	Two author commits; review notes; no secrets

Oral defense: explain why kubectl apply in production is not an acceptable final solution here, and why an Argo application can be Synced while the service is still failing. Each student must answer independently.

Assessment rubric (100 points)

Criterion	Points
Working GitOps application with evidence	15
Drift detected and reconciled	15
Service failure diagnosed and fixed via Git	20
Restricted security policy and safe rollout	20
Readiness failure understanding and recovery	15
Evidence, review and defense	15

Safety / troubleshooting

- Never apply this to a live production cluster; confirm the kind context.
- If Argo CD is not reachable, verify its repo-server and application-controller Pods. Argo setup may require additional time.
- If images cannot be pulled, inspect Pod events and Internet/network configuration.
- If Git push succeeds but Argo has not updated, wait for refresh or annotate the Application to refresh.
- Do not commit real credentials or a GitHub personal-access token.

INSTALLATION APPENDIX | Read before Mission 0

This appendix is part of the student version. It provides the exact preparation, commands, checks and recovery steps needed to run the Kubernetes/GitOps exercise on an isolated local cluster. It does not contain the instructor-only answers.

1. What to install before class

Software / access	Required	Notes
Docker Desktop or Engine	Yes	Running Linux containers; allow ~8 GB memory to Docker/WSL where possible.
kind	Yes	Create a disposable Kubernetes cluster; official: https://kind.sigs.k8s.io/docs/user/quick-start/
kubectl	Yes	Kubernetes client; https://kubernetes.io/docs/tasks/tools/
Git and GitHub account	Yes	Publish a throwaway code-only repository Argo CD can read (public for this lab).
Bash and curl	Yes	All scripts assume Unix shell. Windows: use WSL2 Ubuntu.
Node.js 22+	Optional	Only needed to run local node --test before cluster deployment.
Argo CD CLI	No	The lab uses the Argo CD Kubernetes resources; Argo CD CLI is not required.
Internet	Yes	For cluster image, Argo CD installation, base container image, and Git repo pulls.

2. Installation by operating system

Windows 11 with WSL2 Ubuntu (recommended)

- Install WSL2 Ubuntu: https://learn.microsoft.com/windows/wsl/install ; reboot and create your Linux user account.
- Install Docker Desktop: https://docs.docker.com/desktop/setup/install/windows-install/ ; enable WSL2-based engine and WSL integration for Ubuntu. Start Docker Desktop.
- Inside Ubuntu: sudo apt update && sudo apt install -y git curl bash ca-certificates. Follow the official kind and kubectl pages to install Linux binaries or packages inside WSL2.
- Prefer the Linux home directory, e.g. ~/citybike-k8s-gitops, not a shared /mnt/c/... directory, for Git operations and executable scripts.

macOS

- Install and launch Docker Desktop. Install Git, kind and kubectl with Homebrew if available: brew install git kind kubectl. curl and Bash are typically present.

Ubuntu Linux

- Install Docker Engine following https://docs.docker.com/engine/install/ubuntu/ ; start it and ensure the current user can run docker info.
- Install git/curl/bash: sudo apt update && sudo apt install -y git curl bash ca-certificates. Install kind and kubectl from their official instructions; confirm the executable is in PATH.

3. One-minute tool verification (before the 200-minute clock)

```
git --version
docker --version
docker info
kind version
kubectl version --client
curl --version
bash scripts/preflight.sh
```

If any check fails, do not start the timed exercise. Running Docker Desktop is not enough: docker info must actually connect to the engine.

4. Prepare the GitHub repository (code only)

Extract the entire student ZIP and open the CityBike_Kubernetes_GitOps_Starter folder in a Bash terminal. Confirm that gitops/, bootstrap/, docs/, scripts/, tests/ and kind-config.yaml are present. Create a NEW EMPTY public repository named citybike-k8s-gitops in your own GitHub account. Do not add a GitHub-generated README, .gitignore or license.

```
git init -b main
git config user.name "Your Name"
git config user.email "class-safe@example.com"
git add .
git commit -m "Initialize CityBike GitOps lab"
git remote add origin https://github.com/YOUR_USERNAME/citybike-k8s-gitops.git
git push -u origin main
```

Replace YOUR_USERNAME. If GitHub asks to authenticate, use browser/Git Credential Manager or an appropriate token manager; never paste a token into command arguments, course documents or committed files. Public is required only because this lab does not configure private Argo CD repository credentials.

5. Create the disposable cluster and install Argo CD

```
kind create cluster --name citybike-gitops --config kind-config.yaml --wait 180s
kubectl config current-context
# MUST display: kind-citybike-gitops
kubectl cluster-info
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deployment/argocd-repo-server --timeout=300s
kubectl -n argocd get pods
```

The official Argo CD manifests are downloaded from the internet. A bootstrap can take several minutes. If a namespace already exists, “AlreadyExists” is harmless; proceed with kubectl apply. Use a throwaway cluster only - never use a work or shared production kubeconfig.

6. Bind Argo CD to your Git repository

```
cp bootstrap/application.template.yaml bootstrap/application.yaml
# Edit bootstrap/application.yaml: YOUR_USERNAME -> your account name
# Confirm repoURL and targetRevision: main
kubectl apply -f bootstrap/application.yaml
kubectl -n argocd get applications.argoproj.io citybike
kubectl -n citybike get deployment,pods,service
bash scripts/smoke-service.sh
```

Do not apply the template before replacing YOUR_USERNAME. Argo CD must be able to fetch the repository. Expect two Ready replicas in the initial healthy state and a successful Service smoke test. Synced and Healthy may require time to appear; investigate image pulls, Argo repo access and resources rather than assuming instant readiness.

7. Optional local application tests

```
node --version  # Node.js 22+ recommended
node --test tests/server.test.js
kubectl kustomize gitops
```

These commands check the Node application and manifest rendering only. They do NOT prove that Argo CD deployed correctly or that Kubernetes networking and admission policies are correct.

8. Mission troubleshooting playbook

Problem	Evidence / first action
Docker or kind will not start	Check Docker Desktop resources, docker info, disk space and WSL integration.
kubectl acts on the wrong cluster	STOP; kubectl config current-context must be kind-citybike-gitops.
Argo CD pods Pending / ImagePullBackOff	kubectl -n argocd get pods; describe failing Pod; check network.
Application OutOfSync or Unknown	Check public Git repo URL and branch, source path gitops, repo-server logs; refresh Application.
App Synced but smoke fails	Check Service selector/targetPort, endpoints and Pod readiness; see Mission 3.
Pod creation rejected by Restricted	Inspect kubectl describe output; fix securityContext in Git, do not disable policy.
Readiness rollout hangs	Inspect kubectl get pods, describe deployment and the readiness endpoint; fix in Git.
Smoke script says port busy	Stop the old port-forward, or set PORT=18081 when running script.
Git push done but Argo unchanged	Wait for the next refresh, then request a hard Application refresh if needed.

9. After class: cleanup

Only after confirming the kind context and saving evidence:

```
kubectl config current-context
kind delete cluster --name citybike-gitops
```

Optionally remove the throwaway GitHub repo from GitHub settings when no longer needed; do not delete shared or production environments.

10. Minimum proof before students begin Mission 1

☐ Docker daemon responds; kind and kubectl are installed.

☐ kubectl context is kind-citybike-gitops.

☐ Argo CD repo-server is running.

☐ Public code-only GitHub repository is reachable.

☐ Argo Application points to the correct repository and gitops/ path.

☐ Two initial Pods become Ready and the Service smoke test passes.

Technical qualification: application unit tests, YAML parsing and shell syntax were checked during pack development, but real cluster + Argo CD execution was not possible in this environment. Instructor or students must run the full preflight on a machine with Docker/kind/kubectl before claiming end-to-end validation.